

PucePark — Computación en la Nube

Proyecto Integrador P02 · PUCE TEC Despliegue del sistema en entornos virtualizados (Docker) con arquitectura multicapa.

1. Conceptos de computación en la nube (criterio 4.1)

Computación en la nube: entrega de recursos de cómputo (servidores, almacenamiento, red, bases de datos) bajo demanda a través de Internet, con pago por uso y aprovisionamiento elástico.

Modelos de servicio: - **IaaS** (Infraestructura como Servicio): máquinas virtuales, redes y almacenamiento. *PucePark se despliega sobre este modelo: contenedores Docker que pueden correr en una VM/instancia IaaS (EC2, Compute Engine, etc.).* - **PaaS:** plataforma gestionada (runtime + BD administrada). - **SaaS:** software final consumido por el usuario. *La app PucePark sería el SaaS resultante.*

Virtualización vs contenedores: - *Virtualización (VM):* cada instancia incluye su propio SO → mayor aislamiento, más peso. - *Contenedores (Docker):* comparten el kernel del host → arranque rápido, menor consumo, ideales para microservicios. **PucePark usa contenedores.**

Escalamiento vertical vs horizontal

Aspecto	Escalamiento Vertical (scale-up)	Escalamiento Horizontal (scale-out)
Qué hace	Añadir más recursos (CPU/RAM) a la misma máquina	Añadir más instancias de la aplicación
Ventajas	Simple, sin cambios de arquitectura; sin sincronización entre nodos	Alta disponibilidad, tolerancia a fallos, crecimiento casi ilimitado, balanceo de carga
Desventajas	Límite físico del hardware; punto único de fallo; downtime al escalar	Requiere servicios <i>stateless</i> y balanceador; mayor complejidad operativa
En PucePark	Subir CPU/RAM al contenedor <code>park-app</code> si una zona recibe mucha carga	Levantar N réplicas de <code>park-app / users-service</code> detrás de nginx (balanceo), porque son <i>stateless</i> (el estado vive en Postgres y en el JWT)

Conclusión aplicada: PucePark está diseñado para **escalamiento horizontal**: los microservicios no guardan estado en memoria (la sesión es el JWT de Cognito y los datos están en la BD), por lo que se pueden replicar y poner detrás de nginx como balanceador. La base de datos escalaría verticalmente o con réplicas de lectura.

2. Virtualización / contenedores (criterio 4.2)

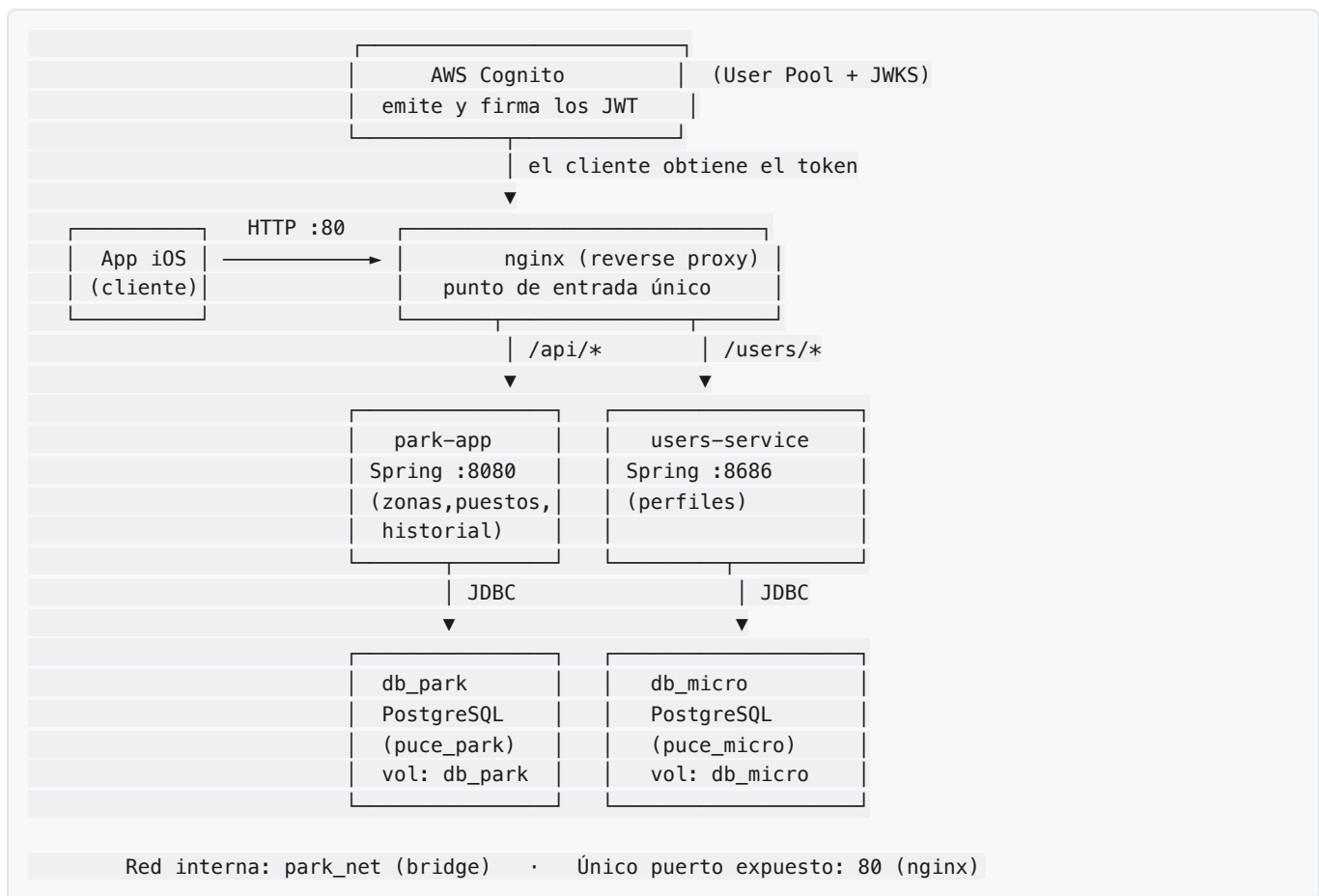
Sistema **multicapa y multiplataforma** orquestado con `docker-compose` (5 contenedores):

Servicio	Imagen/Build	Rol	Puerto
<code>nginx</code>	<code>nginx:alpine</code>	Reverse proxy + panel web <code>/admin/</code> · punto de entrada único	80 → host
<code>park-app</code>	build (Spring Boot/Kotlin)	API de parqueo (<code>/api/*</code>)	8080 (interno)

Servicio	Imagen/Build	Rol	Puerto
users-service	build (Spring Boot/Kotlin)	Microservicio de perfiles (/users/*)	8686 (interno)
db_park	postgres:16-alpine	BD del parqueo (puce_park)	5434 → host
db_micro	postgres:16-alpine	BD de usuarios (puce_micro)	5435 → host

Características de infraestructura implementadas: - **Red privada** park_net (bridge) — los servicios se comunican por nombre de contenedor. - **Volúmenes persistentes** db_park_data, db_micro_data — los datos sobreviven a reinicios. - **Healthchecks** en las BDs (pg_isready) + depends_on: condition: service_healthy. - **Multi-stage build** en el Dockerfile (compila con Gradle y empaqueta solo el JRE + jar). - **Punto de entrada único**: solo nginx publica puerto al host; los servicios usan expose.

3. Diseño de arquitectura de infraestructura (criterio 4.3)



Principios de diseño: - **Aislamiento por servicio**: cada microservicio tiene su BD; no comparten esquema ni hacen *joins* cruzados. - **Sin llamadas entre servicios**: la confianza se delega al JWT de Cognito (cada servicio valida por su cuenta). - **Seguridad perimetral**: solo nginx queda expuesto; las BDs y servicios no son accesibles directamente desde fuera (los puertos 5434/5435 se publican solo para inspección en desarrollo). - **Preparado para la nube**: este docker-compose puede desplegarse en una instancia **IaaS** (AWS EC2 / GCP Compute Engine) tal cual, o migrarse a un orquestador (ECS/Kubernetes) para escalamiento horizontal real replicando park-app / users-service.

4. Despliegue realizado en AWS (evidencia · criterio 4.3)

El sistema **no solo está preparado** para la nube: se desplegó efectivamente en **AWS**.

Backend en EC2 (IaaS)

- Instancia **Amazon EC2** (Amazon Linux 2023, `t3.small`) — modelo **IaaS**.
- Arranque automatizado con **User Data** (`cloud/ec2/user-data.sh`): instala Docker + Compose, clona el repositorio y ejecuta `docker compose up -d --build` (los 5 contenedores).
- **Security Group**: solo `80` (HTTP público) y `22` (SSH restringido a la IP del equipo); las BDs quedan internas.
- Verificación en vivo:
 - `http://<IP-EC2>/api/v1/zonas` → **401** (park-app responde, requiere token).
 - `http://<IP-EC2>/users/me` → **401** (users-service responde).
 - `http://<IP-EC2>/admin/` → **panel web de administración** servido desde la nube.
- Guía completa: `cloud/ec2/README-ec2.md`.

Página de descarga en S3 (almacenamiento en la nube)

- Bucket **Amazon S3** configurado como *static website hosting* con la página de descarga de la app (iOS/APK).
- Guía: `cloud/README-despliegue-s3.md`.

Panel web de administración (cliente adicional en la nube)

El mismo nginx sirve, en `/admin/`, un panel web estático donde el **administrador** inicia sesión con Cognito y gestiona zonas, puestos, ranking e historial — demostrando **un backend, dos clientes** (app iOS + panel web) sobre la misma infraestructura.

5. Ventajas del enfoque para la nube

- **Reproducibilidad**: `docker-compose up -d --build` levanta todo el sistema en cualquier host con Docker (probado en local y en EC2).
- **Elasticidad**: servicios stateless → réplicas horizontales bajo nginx.
- **Costo**: contenedores livianos (alpine) → menor consumo que VMs completas.
- **Portabilidad multiplataforma**: mismas imágenes en local, IaaS o nube pública.